

CSC 452/552 Operations Systems

Project 4 Buddy System

Name: Amanda Stroud

Bronco ID: 114160989

Date: 3 December 2024

1. Project Overview

This project is designed to implement a memory manager with the buddy system. It will take a specified size for the pool and can be filled based on those parameters.

2. Project Management Plan

a) Task 3 (Implement Buddy) :

Task 3 was the bulk of the project and my approach was to implement all the functions first before I tested each one. The buddy init, buddy destroy, and btok was all worked on in class. I took time to study what buddy init does as this helps within the project later. It sets up the pool's kval_m and num of bytes, and uses mmap to get the base location. Then we just fill up from 0 to kval_m - 1. Then we set the base address at the location of kval_m for next and previous and set this as block available. For destroy, this sets munmap to point at an invalid location. I had to adjust btok a little bit from what we did in class, just that when bytes = 1 we return 0, this helps account for the 0 to kval_m inclusive.

The first one I did was Buddy_Calc, the method stated that we needed to find the buddy relative to the base address. So I subtracted the block address by the base address, and the paper mentions the xor operator relative to 2^k , so we can xor the retrieved address by 2^k . Then the base plus the buddy address is returned. I had to go back to this one a few times adjusting different values to get the right offset values with the base address since the paper was a bit vague.

Then I went and did Buddy_Malloc and this was definitely one of the hardest methods to get working. It took me a while to wrap my head around how malloc worked doing it by hand. I didn't really focus on the buddy paper until later when I realized it gave the specifics on how the method worked and the lab website has

a chart that shows the conversions of variables to the variables within my C code. I will discuss more on that in the self reflection. But when I did refer to the paper I followed each step, R1 has you check for `pool->avail[k].next != &pool->avail[k]` to check for an available spot, and R2 has it set the L to `pool->avail[k].next` at this condition. This is the start of removing the found L from the list, by setting up the buddy, and setting L as reserved. Then if no split (R3) then the algorithm ends, if there is a split then the algorithm goes through a while loop based on the R3 condition (so while $j > k$) we decrement j, and add the unused half to the available list. Finally the return is the address of L. I also wasn't entirely sure of the logic for why I needed to return $L + 1$, but one of the test cases accounts for that so I made it like that and it seems to work. The only thing I had to do was decrement by 1 for the address in the buddy free and it seems to work great.

For the buddy free I also went off the paper, I didn't have to change as much for what I did not working off the paper, but the paper had three conditions for buddy free. First in S1 we set the buddy value of P to what the `buddy_calc` returns of L, I also put this within the loop since we want to calculate the buddies for all values that need to be freed while the kval in L is less than kval_m for the condition of $k == m$ going to S3. Now if there are buddies we combine based on the variables set in S2, increment the k in L, and if $P < L$, then $L = P$, and do that based on the condition set in S1. For S3 the equations used there didn't seem to work, but then I remembered that step R4 from buddy malloc adds the buddy to the list so I did this but with L instead.

Finally `Buddy_Realloc` was surprisingly straight forward, I feel like the method description guided me pretty well. Pretty much I just used `buddy_malloc` to create a new allocated space, copy the original data based on 2^k with `memcpy`, and then free the original data.

b) Task 4:

Task 4 was pretty easy, I replicated the other test structures and checked for multiple allocated memory within one pool to make sure that I can add both and then free both of them to be freed properly. I also checked for too big of values, null values (such as

if pool wasn't initialized), 0 bytes, reallocating memory into a bigger, smaller, and attempting to allocate the same size (just returns the original address without changing anything). I am assuming I got most test cases but luckily it all seems successful.

```
@Stroamanda →/workspaces/cs452-project-starter (p4-real) $ make check
ASAN_OPTIONS=detect_leaks=1 ./test-lab
Random seed:1733112957
Running memory tests.
->Testing buddy init
tests/test-lab.c:319:test_buddy_init:PASS
->Test allocating and freeing 1 byte
tests/test-lab.c:320:test_buddy_malloc_one_byte:PASS
->Testing size that will consume entire memory pool
tests/test-lab.c:321:test_buddy_malloc_one_large:PASS
->Test allocating and freeing 5 bytes
tests/test-lab.c:322:test_buddy_malloc_five_bytes:PASS
->Test allocating and freeing 5 and 3 bytes
tests/test-lab.c:323:test_buddy_malloc_five_three_bytes:PASS
->Test allocating 30000000 bytes to be NULL
tests/test-lab.c:324:test_buddy_malloc_too_large:PASS
->Test allocating 0 bytes to be NULL
tests/test-lab.c:325:test_buddy_malloc_zero:PASS
->Test allocating 5 bytes and reallocating to 16 bytes
tests/test-lab.c:326:test_buddy_realloc_5_to_16:PASS
->Test allocating 5 bytes and reallocating to 4 bytes
tests/test-lab.c:327:test_buddy_realloc_5_to_4:PASS
->Test for uninitialized pool
tests/test-lab.c:328:test_buddy_malloc_uninitialized_pool:PASS
->Test allocating 5 bytes and reallocating to same size
tests/test-lab.c:329:test_buddy_realloc_same_size:PASS
->Test allocating 5 bytes and reallocating to 0
tests/test-lab.c:330:test_buddy_realloc_0:PASS
```

3. Project Deliveries

a) How to compile and use my code?

To compile, you can run `make clean`, `make`, and `make check`, which will run all the unit tests that were initially there as well as the custom ones I added.

b) Any self-modification?

I included extra unit tests to check for more conditions that weren't originally checked for.

c) Summary of Results.

My results seem to be accurate, it passes all the tests I included, and checks for all sorts of mallocs, and reallocations, and freeing for each one that can be freed.

4. Self-Reflection of Project

This was probably one of the hardest projects I have done in all my CS

classes. I have had pretty difficult projects in the past but this one was pretty up there. I am glad I decided to start and finish it before the deadline. One thing I wish I would've utilized more was the Buddy System article provided. I did a basic read over it but I didn't know exactly how to translate it until I saw a conversion chart on the project page. This helped so much and helped me fix a ton of bugs I had been staring at for hours. One thing I did at first is that I thought we were grabbing memory locations from the avail list itself, so I ended up looping through the avail list and was trying to see when a block was available. I was also confused between block unused and block available, so I checked for both conditions but then this would just malloc memory for that specific space and not even use the buddy system like intended. But even with having code that wasn't really lined up with the section on Buddy Systems, it got me used to stepping through and debugging in vscode which helped a ton, I could see how values changed and be able to see how the memory itself worked, which helped solve initial bugs. So after including the logic found within the Buddy System pdf itself, I was able to fix up any issues I have and now I understand how this system works quite a bit when before I was pretty lost on how the whole thing worked. It makes more sense that we split based on the max memory location that is available first to be able to use the buddy system, and not just going to each individual one. While this project was very frustrating, I learned a lot from it and it was nice to finally be able to get it running properly. It's also started to become easier to read from CS textbooks, and I wish I would've read through it a bit more to avoid a lot of the issues I ran into, though I think tinkering with it first helped me understand the problem and how to properly implement what was shown in the textbook.

5. Comments for Project (optional)

Never take a provided textbook section in a project for granted!

<https://shanepanter.com/articles/Buddy-System.pdf>

6. Use of AI for debugging (optional)

The one thing I did use AI for was how to calculate the memories together since these are of type struct avail * and wasn't entirely sure how to get it to calculate together to work. I was running into a bug of it not being calculated the way it needed to, so this helped me fix that issue.

how can I add two memory addresses together of type struct avail * with size_t?



In C, memory addresses of type `struct avail *` and `size_t` (which typically represents a byte offset) cannot be directly added in the way that two integers can. However, you can perform pointer arithmetic to add an offset of type `size_t` to a pointer of type `struct avail *`.

To add a `size_t` value to a `struct avail *` pointer, you would do the following:

```
c Copy code  
  
struct avail *ptr; // Pointer to struct avail  
size_t offset;    // Size offset in bytes  
  
struct avail *new_ptr = (struct avail *)((char *)ptr + offset);
```

Explanation:

1. **Pointer Arithmetic:** When you add a `size_t` value to a pointer, C assumes the offset is in terms of the size of the type the pointer points to. So, to explicitly treat the offset in bytes, you need to cast the pointer to `char *` (since a `char` is 1 byte in size), and then add the `size_t` offset.
2. **Casting Back:** After adding the offset, you cast the result back to the original pointer type, in this case, `struct avail *`.

This will correctly move the pointer by the given number of bytes (`offset`).